

2.11 - Contêineres: stateless e imutável

Por que um arquivo no disco quebra o sistema

404 em metade dos downloads

- **Upload de comprovantes salvava arquivo no disco local**
 - No scale-out com três instâncias, metade dava 404

LUNALOG

A LunaLog containerizou os serviços. Na primeira tentativa, o serviço de upload de comprovantes guardava os arquivos no filesystem local do contêiner. No primeiro scale-out, com três instâncias rodando, metade das requisições de download retornava 404. O arquivo tinha sido salvo em uma instância, e a requisição caiu em outra. O time aprendeu na prática: estado local em contêiner é garantia de problema em ambiente distribuído.

O modelo de imagem imutável

Construa uma vez, rode em qualquer lugar

- **A imagem do contêiner não muda após ser construída**
 - A mesma imagem roda igual em qualquer ambiente
- **Configuração e segredos entram por fora, em runtime**
 - Variáveis de ambiente e cofres, nunca no código
- **Atualizar é trocar a imagem, não editar a viva**
 - Rollback vira voltar para a imagem anterior
- **Imutabilidade elimina o servidor floco de neve**
 - Acabam as diferenças misteriosas entre ambientes

Externalizar estado é o pré-requisito

- Imagem imutável obriga a tirar o estado de dentro
- Estado, config e segredos moram fora do contêiner

Se a imagem é imutável e descartável, qualquer coisa que precise sobreviver a um restart precisa estar fora dela. Estado em banco ou storage, config em variável, segredo em cofre.

- O que mora dentro do contêiner é descartável por design

Stateless como pré-requisito de escala

- **Serviço stateless escala na horizontal sem dor**
 - Qualquer instância atende qualquer requisição
- **Sessão e arquivos vão para storage externo**
 - S3, banco ou cache compartilhado entre instâncias
- **Health check é um contrato com o orquestrador**
 - Pod que falha no check é substituído sem intervenção
- **Kubernetes assume escala, rede e reinício**
 - O cluster cuida do ciclo de vida dos pods

Stateful vs. stateless no cluster

Serviço stateful (problemático)

- Guarda arquivo ou sessão no disco local
- Cada instância tem dados diferentes
- Scale-out quebra o que depende do local
- Reinício do pod perde dados

Serviço stateless (escalável)

- Nenhum estado preso a uma instância
- Todas as instâncias são intercambiáveis
- Scale-out é transparente para o cliente
- Reinício do pod não perde nada



Contêineres stateless são o alicerce. Mas existe um repertório de padrões que operam dentro do próprio pod, e que resolvem problemas de infraestrutura sem tocar em uma linha do código da aplicação. Como adicionar coleta de logs ou um proxy de rede a doze serviços de uma vez, sem reescrever nenhum deles?

Padrões de contêineres: sidecar, ambassador e adapter